

Package ‘facereaderconverter’

September 24, 2026

Title Process of FaceReader outputs and emotional episodes

Version 0.3.2

Description Tools to convert FaceReader outputs; Markup of emotional episodes.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

Depends R (>= 4.1.0)

Imports dplyr,

hms,
janitor,
readr,
readxl,
stringr,
tidyselect,
tools,
tibble,
Rcpp,
data.table,
tidyr

LinkingTo Rcpp

Suggests openxlsx,
testthat (>= 3.0.0)

Config/testthat/edition 3

Config/roxygen2/version 8.1.0

Contents

add_delta_column	2
convertFRDirectory	3
convertFRExcelFiles	4
convertFRFiles	5
convert_to_episodes	6
delta	8
delta_episodes	9
export_shared_synchrony_clips	9
extract_id_subject_metadata	11

extract_subject_id_metadata	12
loadFRfile	13
locf	14
map_paths	15
negative_controls	15
parse_time_to_frame	17
reaction_rate	17
reaction_rate_by_episode	19
shared_synchronous_episodes	20
synchrony	22
synchrony_by_episode	23
synchrony_moments_pipeline	25
to_seconds	27

Index	28
--------------	-----------

add_delta_column	<i>Add delta column to coding_df</i>
------------------	--------------------------------------

Description

add_delta_column() is deprecated because convert_to_episodes() now computes and returns delta automatically.

Usage

```
add_delta_column(
  coding,
  delta = 0.1,
  delta_window = 0.2,
  fps = 30L,
  cores = 0L
)
```

Arguments

coding	Data frame with columns: id, subject, emotion, frame (or video_time), value, or an fr_coding object.
delta	Threshold for both up and down.
delta_window	Window size in seconds.
fps	Frames per second.
cores	integer Number of threads to use. Default 0 is auto.

Value

coding_df with a delta column where 1 means up and 0 means down.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
add_delta_column(
  coding_df,
  delta_window = 0.2,
  delta = 0.1,
  fps = 30
)

## End(Not run)
```

convertFRDirectory	<i>Convert a directory of Facereader files to CSV</i>
--------------------	---

Description

Reads TXT, XLSX, and CSV files in a folder and converts them to CSV.

Usage

```
convertFRDirectory(
  inpath,
  outpath = inpath,
  recursive = TRUE,
  pattern = NULL,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  save_metadata = outpath,
  metadata_filename = "metadata.csv",
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  cores = 0L,
  ...,
  id = NULL,
  subject = NULL
)
```

Arguments

inpath	Path to an existing directory.
outpath	Path to save the csvs to defaults to the inpath
recursive	Bool as to whether to look for all files in directory (TRUE) or just the root folder (FALSE)
pattern	a regex pattern of files to test, if NULL then will look for TXT, XLSX, and CSV files
values_as_numeric	Save values as numeric, where applicable
clean_names	returns janitor-style clean names

save_metadata save the metadata as a csv in the outpath, set to NULL to not save
 metadata_filename filename of the metadata csv
 fail_codes adds a column with the fail reason, True or False. Column then has 0 for success,
 1 for fit_failed, 2 for find_failed
 duplicate_timecodes_as_error throws an error if there are duplicate timecodes, if FALSE then throws warning
 cores integer Number of threads to use. Default 0 is auto.
 ... arguments passed as necessary
 id Optional scalar ID or function of each input path returning one.
 subject Optional scalar subject or function of each input path returning one.

Value

Invisibly returns the metadata.

Examples

```
## Not run:
convertFRDirectory(
  inpath="directory_of_txt_files",
  outpath="directory_to_save_csvs_to",
  values_as_numeric = TRUE
)

## End(Not run)
```

convertFRExcelFiles *Load FaceReader Excel exports*

Description

Reads a FaceReader .xlsx file, detects the header row automatically, and returns the parsed data frame. The function follows the same post-processing conventions as convertFRFiles() where applicable.

Usage

```
convertFRExcelFiles(
  inpath,
  return_data = TRUE,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  sheet = 1,
  ...,
  id = NULL,
  subject = NULL
)
```

Arguments

inpath	Path to an existing .xlsx file.
return_data	Return the parsed data instead of metadata.
values_as_numeric	Convert Video Time to hms and detailed values to numeric.
clean_names	Apply janitor::clean_names() to the data.
fail_codes	Add a fail_code column for detailed exports.
duplicate_timecodes_as_error	Throw an error if duplicate timecodes are found.
sheet	Excel sheet to read. Defaults to the first sheet.
...	Additional arguments passed to janitor::clean_names().
id	Optional scalar ID or function of inpath returning one.
subject	Optional scalar subject or function of inpath returning one.

Value

Invisibly returns the parsed data when return_data = TRUE; otherwise returns metadata about the import.

Examples

```
## Not run:
convertFRExcelFiles(
  inpath = "FaceReaderOutput.xlsx",
  return_data = TRUE,
  values_as_numeric = TRUE
)

## End(Not run)
```

convertFRFiles	<i>Convert Facereader TXT to CSV</i>
----------------	--------------------------------------

Description

Reads a .txt file, detects the FaceReader header row automatically, guesses the delimiter from the header line, writes a .csv with the same basename in the same directory, and returns the path to the created CSV (invisibly).

Usage

```
convertFRFiles(
  inpath,
  outpath = paste0(tools::file_path_sans_ext(inpath), ".csv"),
  return_data = FALSE,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
```

```

duplicate_timecodes_as_error = TRUE,
...,
id = NULL,
subject = NULL
)

```

Arguments

inpath	Path to an existing .txt file.
outpath	Path to save the csv to defaults to the inpath
return_data	Bool to return the data from the txt rather than the metadata
values_as_numeric	Save values as numeric, where applicable
clean_names	returns janitor-style clean names
fail_codes	adds a column with the fail reason, True or False. Column then has 0 for success, 1 for fit_failed, 2 for find_failed
duplicate_timecodes_as_error	throws an error if there are duplicate timecodes, if FALSE then throws warning
...	arguments passed as necessary
id	Optional scalar ID or function of inpath returning one.
subject	Optional scalar subject or function of inpath returning one.

Value

Invisibly returns the metadata.

Examples

```

## Not run:
convertFRFiles(
  inpath="FaceReaderOutput.txt",
  values_as_numeric = TRUE
)

## End(Not run)

```

convert_to_episodes	<i>Convert coding_df to episodes</i>
---------------------	--------------------------------------

Description

Convert coding_df to episodes

Usage

```
convert_to_episodes(
  coding_df,
  T_up = 0.2,
  T_down = 0.1,
  delta = 0.1,
  delta_window = 0.2,
  min_dur_sec = 0.1,
  consecutive_missing = 150L,
  fps = 30L,
  cores = 0L
)
```

Arguments

<code>coding_df</code>	a dataframe or otherwise from a FaceReader output. <code>id</code> and <code>subject</code> should be present.
<code>T_up</code>	numeric Upper threshold for entering an episode. Default: 0.2.
<code>T_down</code>	numeric Lower threshold for exiting an episode. Default: 0.1.
<code>delta</code>	numeric Threshold used when computing the returned <code>delta</code> column, as the minimum k-step change required. It no longer controls episode boundaries. Default: 0.1.
<code>delta_window</code>	numeric Time window (in seconds) used to derive <code>k</code> , the lag for the returned k-step difference <code>delta</code> column. It no longer controls episode boundaries. Default: 0.1.
<code>min_dur_sec</code>	numeric Minimum episode duration (in seconds). Default: 0.1.
<code>consecutive_missing</code>	integer Maximum allowed consecutive missing (NA) frames while in-state before forcing episode end. Default: 150L.
<code>fps</code>	integer Frames per second (sampling rate of the data). Default: 30L.
<code>cores</code>	integer Number of threads to use. Default 0 is auto.

Details

Episode boundaries are determined only by `T_up` and `T_down`. The returned `delta` column is computed separately and delta-up rows are also returned in `deltas` for downstream functions such as `reaction_rate()`. The function uses the exported native binding `hysteresis_state` if available; otherwise it will error. It relies on **data.table** for fast grouping and joins.

Value

A list with four elements:

- episodes** `data.table` of detected episodes with columns `start_frame`, `end_frame`, `n_frames`, `duration_s`, `id`, `subject`, `emotion`, `start_time`, `end_time`, `run_id`, and `max_value`.
- deltas** `data.table` of delta-up reaction events with the same columns as episodes, except `delta_id` replaces `run_id`, and with `max_delta`, the value range from one delta window before the event start through the event end.

coding Annotated data.table containing the original columns plus id, subject, emotion, value, delta, delta_id, run_id, status, and in_state. status marks episode boundaries with 1L at the start frame and 0L at the end frame; in_state is TRUE for frames inside detected episodes.

metadata Metadata used to create the returned object.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
convert_to_episodes(coding_df)

## End(Not run)
```

delta	<i>Add delta column alias</i>
-------	-------------------------------

Description

delta() is deprecated because convert_to_episodes() now computes and returns delta automatically.

Usage

```
delta(coding, delta = 0.1, delta_window = 0.2, fps = 30L, cores = 0L)
```

Arguments

coding	Data frame with columns: id, subject, emotion, frame (or video_time), value, or an fr_coding object.
delta	Threshold for both up and down.
delta_window	Window size in seconds.
fps	Frames per second.
cores	integer Number of threads to use. Default 0 is auto.

Value

coding_df with a delta column where 1 means up and 0 means down.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
delta(
  coding_df,
  delta_window = 0.2,
  delta = 0.1,
  fps = 30
)

## End(Not run)
```

delta_episodes	<i>Add create delta episodes</i>
----------------	----------------------------------

Description

Add create delta episodes

Usage

```
delta_episodes(coding, fps = 30L, cores = 0L)
```

Arguments

coding	Data frame with columns: id, subject, emotion, frame (or video_time), value, or an fr_coding object
fps	Frames per second
cores	integer Number of threads to use. Default 0 is auto.

Value

episodes data.table of detected episodes with columns start_frame, end_frame, start_time, end_time, n_frames, duration_s, id, subject, emotion, and run_id.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
x=add_delta_column(
  coding_df,
  delta_window = 0.2,
  delta = 0.1,
  fps = 30
)
delta_episodes(x)

## End(Not run)
```

export_shared_synchrony_clips	<i>Export clips for shared synchronous episodes</i>
-------------------------------	---

Description

Selects shared synchronous intervals with the largest combined emotion values, or a named subject's highest-valued shared or individual episodes, and exports the corresponding video segments with FFmpeg. Frame ranges are inclusive: a range from frame 0 through frame 0 has duration 1 / fps.

Usage

```

export_shared_synchrony_clips(
  coded_data,
  shared_synchrony,
  video_paths,
  n = 10L,
  emotion = "happy",
  optimised_subject = "both",
  only_synchronies = TRUE,
  buffer = 0,
  buffer_units = c("seconds", "frames"),
  buffer_frames = 0L,
  buffer_seconds = 0,
  output_path = NULL,
  output = c("zip", "folder"),
  overwrite = FALSE,
  ffmpeg = "ffmpeg"
)

```

Arguments

<code>coded_data</code>	A converted <code>fr_coding</code> object with <code>metadata\$fps</code> .
<code>shared_synchrony</code>	A table returned by <code>shared_synchronous_episodes()</code> .
<code>video_paths</code>	Named character vector of source video paths. Names must match values in the <code>shared-synchrony</code> id column.
<code>n</code>	Number of highest-ranked intervals to export. Use 0 to export all intervals remaining after filtering.
<code>emotion</code>	Optional character vector of emotions to retain.
<code>optimised_subject</code>	Subject-name regular expression used to rank clips by one subject's maximum emotion value. Defaults to "both", which ranks shared synchronies by their combined value.
<code>only_synchronies</code>	Logical; when TRUE, select shared synchronies. When FALSE, select individual episodes for <code>optimised_subject</code> . Ignored when <code>optimised_subject = "both"</code> .
<code>buffer</code>	A non-negative number for a symmetric buffer, or a named two-element vector with names before and after for asymmetric buffers. Units are controlled by <code>buffer_units</code> .
<code>buffer_units</code>	Buffer units: "seconds" or "frames".
<code>buffer_frames</code>	Non-negative whole-number symmetric frame buffer. Kept for compatibility; use <code>buffer</code> and <code>buffer_units</code> for new code.
<code>buffer_seconds</code>	Non-negative symmetric time buffer in seconds. Kept for compatibility; use <code>buffer</code> and <code>buffer_units</code> for new code.
<code>output_path</code>	Optional ZIP file path when <code>output = "zip"</code> , or output directory when <code>output = "folder"</code> . By default, output is placed beside the selected source video. All selected videos must then be in the same folder.
<code>output</code>	Output type: "zip" (the default) or "folder".

overwrite	Logical; overwrite an existing ZIP archive or non-empty output folder. Defaults to FALSE.
ffmpeg	Path to the FFmpeg executable or its command name.

Value

A `data.table` manifest invisibly. For ZIP output, the manifest is also included in the archive as `manifest.csv`.

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_data)
shared <- shared_synchronous_episodes(coded_data)
export_shared_synchrony_clips(
  coded_data,
  shared,
  video_paths = c("1" = "recording.mp4"),
  n = 10,
  optimised_subject = "child",
  only_synchronies = FALSE,
  buffer = c(before = 2, after = 1),
  output_path = "recording-shared-clips.zip"
)

## End(Not run)
```

```
extract_id_subject_metadata
```

Extract ID and subject metadata from a FaceReader filename

Description

Compatibility alias for `extract_subject_id_metadata()`.

Usage

```
extract_id_subject_metadata(
  path,
  id_pattern = "(?![0-9])\\s*[0-9]{4}\\s*(?![0-9])",
  subject_pattern = "(?![a-z])(mum|teen)(?![a-z])"
)
```

Arguments

path	Path to a FaceReader file.
id_pattern	Regular expression used to extract the identifier. Defaults to a four-digit identifier with optional surrounding whitespace.
subject_pattern	Regular expression used to extract the subject. Defaults to mum or teen as a standalone word.

Value

A list with character elements `id` and `subject`. Each element is NA when its pattern does not match.

Examples

```
## Not run:
extract_id_subject_metadata("FR9 1218 mum_Analysis_detailed.xlsx")

## End(Not run)
```

```
extract_subject_id_metadata
```

Extract subject and ID metadata from a FaceReader filename

Description

Extracts an identifier and subject label from a file path using caller-supplied regular expressions. The identifier is trimmed after extraction, while the subject is matched case-insensitively.

Usage

```
extract_subject_id_metadata(
  path,
  id_pattern = "(?<![0-9])\\s*[0-9]{4}\\s*(?![0-9])",
  subject_pattern = "(?<![a-z])(mum|teen)(?![a-z])"
)
```

Arguments

<code>path</code>	Path to a FaceReader file.
<code>id_pattern</code>	Regular expression used to extract the identifier. Defaults to a four-digit identifier with optional surrounding whitespace.
<code>subject_pattern</code>	Regular expression used to extract the subject. Defaults to mum or teen as a standalone word.

Value

A list with character elements `id` and `subject`. Each element is NA when its pattern does not match.

Examples

```
## Not run:
extract_subject_id_metadata(
  "8895 mum FR9_00024 mum_Analysis_detailed.xlsx",
  id_pattern = "(?<![0-9])\\s*[0-9]{4}\\s*(?![0-9])",
  subject_pattern = "(?<![a-z])(mum|teen)(?![a-z])"
)

## End(Not run)
```

loadFRfile

*Load FaceReader files into memory***Description**

Reads a FaceReader export file into memory by dispatching to the existing TXT, Excel, or CSV readers based on file extension. The wrapper always returns the parsed data and never writes an output file.

Usage

```
loadFRfile(
  inpath,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  sheet = 1,
  ...,
  id = NULL,
  subject = NULL,
  csv_args = list(),
  clean_names_args = list()
)
```

Arguments

<code>inpath</code>	Path to an existing FaceReader export file.
<code>values_as_numeric</code>	Convert Video Time to hms and detailed values to numeric where applicable.
<code>clean_names</code>	Apply <code>janitor::clean_names()</code> to the data.
<code>fail_codes</code>	Add a <code>fail_code</code> column for detailed exports.
<code>duplicate_timecodes_as_error</code>	Throw an error if duplicate timecodes are found.
<code>sheet</code>	Excel sheet to read when importing .xlsx files.
<code>...</code>	Legacy additional arguments passed to <code>janitor::clean_names()</code> for TXT and XLSX imports and to <code>readr::read_csv()</code> for CSV imports.
<code>id</code>	Optional scalar ID or function of <code>inpath</code> returning one.
<code>subject</code>	Optional scalar subject or function of <code>inpath</code> returning one.
<code>csv_args</code>	Named list of additional arguments for <code>readr::read_csv()</code> .
<code>clean_names_args</code>	Named list of additional arguments for <code>janitor::clean_names()</code> .

Value

Invisibly returns the parsed data frame.

Examples

```
## Not run:
loadFRfile(
  inpath = "FaceReaderOutput.txt",
  values_as_numeric = TRUE
)

## End(Not run)
```

locf

Apply LOCF and Extract Episodes

Description

Applies Last Observation Carried Forward (LOCF) logic to the coding data.table (from `convert_to_episodes`), imputing missing `run_id` values within blocks of non-missing value, grouped by `id`, `subject`, and `emotion`. Returns both the imputed coding table and a summary of contiguous episodes.

Usage

```
locf(coding, fps = 30, consecutive_missing = NULL)
```

Arguments

<code>coding</code>	A datatable, dataframe or <code>fr_coding</code> object as returned by <code>convert_to_episodes</code> , containing columns: <code>id</code> , <code>subject</code> , <code>emotion</code> , <code>video_time</code> , <code>value</code> , and <code>run_id</code> .
<code>fps</code>	Frames per second (sampling rate of the data). Default: 30L or inherited from an <code>fr_coding</code> object
<code>consecutive_missing</code>	Maximum allowed consecutive missing (NA) frames while in-state before forcing episode end. Default: 150L or inherited from an <code>fr_coding</code> object

Value

A list with four elements:

episodes data.table of contiguous episodes after LOCF, with columns `start_frame`, `end_frame`, `n_frames`, `duration_s`, `id`, `subject`, `emotion`, and `run_id`.

deltas data.table of delta-up events, passed through unchanged when the input is an `fr_coding` object.

coding Annotated data.table with LOCF-imputed `run_id`.

metadata Metadata from `fr_coding` object.

map_paths	<i>Map files from an input root to an output root, preserve structure, create dirs.</i>
-----------	---

Description

Map files from an input root to an output root, preserve structure, create dirs.

Usage

```
map_paths(input_dir, output_dir, files)
```

Arguments

input_dir	Character scalar. The source root directory.
output_dir	Character scalar. The destination root directory.
files	Character vector. File paths under input_dir to be remapped.

Value

Character vector of output file paths (same length as files).

negative_controls	<i>Calculate synchrony for matched negative-control intervals</i>
-------------------	---

Description

Samples control intervals with the same inclusive frame length as supplied episodes, then calculates synchrony for the sampled intervals. The episodes argument can be the output of [synchrony_by_episode\(\)](#) or a compatible episode table. Control intervals cannot overlap supplied episodes or previously accepted controls for the same ID.

Usage

```
negative_controls(
  coded_data,
  episodes,
  mutually_exclusive = FALSE,
  subject = "subject",
  id = "id",
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  fps = 30L,
  missing_threshold = 0,
  exclude_emotions = "neutral",
  max_tries = 1000L
)
```

Arguments

coded_data	Output from <code>convert_to_episodes()</code> .
episodes	A data frame containing episode ranges. It must contain the selected ID column, either the selected subject column or denominator, and emotion, run_id, start_frame, and end_frame. Frame bounds are inclusive.
mutually_exclusive	Logical scalar retained for compatibility. Controls cannot overlap supplied episodes for the same ID regardless of this value.
subject	Character scalar giving the subject column. Defaults to "subject".
id	Character scalar giving the ID column. Defaults to "id".
time_limit	Maximum episode-window length in seconds. NULL is allowed when time_limit_frames is supplied or when constraint_method is "episode". Default is 3.
time_limit_frames	Optional maximum episode-window length in frames. If supplied, this takes precedence over time_limit.
constraint_method	Character scalar controlling how the episode window ends: "strict", "episode", "loose", or "frames". Default is "episode".
fps	Frames per second used to convert time_limit to frames. Default is 30.
missing_threshold	Numeric scalar in $[0, 1]$. Denominator and numerator episodes are kept only when the comparison subject is present for at least this proportion of frames within the episode. Default is 0.
exclude_emotions	Character vector of emotions to exclude from the denominator calculation. Default is "neutral".
max_tries	Maximum number of random control starts attempted for each episode. Defaults to 1000.

Details

When coded_data contains metadata, its metadata\$fps value is used by the synchrony calculations, including when fps is supplied.

Value

A data.table containing the usual `synchrony_by_episode()` columns for matched controls, plus source and control identifiers, frame bounds, attempt count, and matching status. Unmatched source episodes have missing synchrony fields and control_status = "unmatched".

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_data)
episodes <- synchrony_by_episode(coded_data)
negative_controls(coded_data, episodes)

## End(Not run)
```

parse_time_to_frame	<i>Parse video time to frame index</i>
---------------------	--

Description

Convert a time string into a frame index given frames-per-second (fps). Supports "HH:MM:SS", "MM:SS", or plain seconds and is vectorised over video_time.

Usage

```
parse_time_to_frame(video_time, fps)
```

Arguments

video_time	Character or numeric. Time strings in "HH:MM:SS", "MM:SS", or numeric seconds.
fps	Numeric. Frames per second.

Value

Integer vector. Frame indices computed as $\text{round}(\text{seconds} * \text{fps})$.

Examples

```
parse_time_to_frame("00:01:23.5", fps = 30)
parse_time_to_frame("1:23.5", fps = 30)
parse_time_to_frame("83.5", fps = 30)
```

reaction_rate	<i>Calculate reaction rate from converted episodes</i>
---------------	--

Description

Reaction rate mirrors synchrony() but uses the numerator subject's delta-up events as the reaction signal. Denominator episodes come from convert_to_episodes(), and a reaction is counted when the numerator subject has at least one delta event within the constrained denominator episode window.

Usage

```
reaction_rate(
  coded_data,
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  exclude_start = 0.1,
  exclude_start_frames = NULL,
  fps = 30L,
  subject_names = NULL,
  exclude_emotions = "neutral",
  minimum_threshold = 0
)
```

Arguments

<code>coded_data</code>	Output from <code>convert_to_episodes()</code> , which includes frame-level delta values in <code>coded_data\$coding</code> and a reaction-event table in <code>coded_data\$deltas</code> .
<code>time_limit</code>	Maximum episode-window length in seconds when a frame constraint is applied.
<code>time_limit_frames</code>	Optional maximum episode-window length in frames. If supplied, this takes precedence over <code>time_limit</code> .
<code>constraint_method</code>	Character scalar controlling how the reaction window ends. "strict" uses the earlier of the observed episode end and the requested time limit. "episode" uses the observed episode end and ignores <code>time_limit</code> . "loose" uses the later of the observed episode end and the requested time limit. "frames" uses only the requested time limit and ignores the observed episode end. Default is "episode".
<code>exclude_start</code>	Minimum delay, in seconds from the denominator episode start, before numerator <code>delta == 1</code> values count as a reaction.
<code>exclude_start_frames</code>	Optional minimum delay in frames. If supplied, this takes precedence over <code>exclude_start</code> .
<code>fps</code>	Frames per second used to convert between seconds and frames when <code>coded_data\$metadata\$fps</code> is unavailable.
<code>subject_names</code>	Character vector of subject labels to compare. If NULL, all unique subjects in <code>coded_data\$coding</code> are used.
<code>exclude_emotions</code>	Character vector of emotions to exclude from the denominator calculation. Default is "neutral".
<code>minimum_threshold</code>	Numeric scalar in $[0, 1]$. Denominator episodes are kept only when at least this proportion of numerator frames in the constrained window have non-missing values. Default is 0.

Value

A `data.table` with columns `id`, `denominator`, `numerator`, `emotion`, `n_episodes`, `n_reactions`, and `reaction_rate`.

See Also

[reaction_rate_by_episode\(\)](#)

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_df)
reaction_rate(coded_data)

## End(Not run)
```

 reaction_rate_by_episode

Calculate reaction rate by denominator episode

Description

Calculate reaction rate by denominator episode

Usage

```
reaction_rate_by_episode(
  coded_data,
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  exclude_start = 0.1,
  exclude_start_frames = NULL,
  fps = 30L,
  subject_names = NULL,
  exclude_emotions = "neutral",
  minimum_threshold = 0
)
```

Arguments

coded_data	Output from <code>convert_to_episodes()</code> , which includes frame-level delta values in <code>coded_data\$coding</code> and a reaction-event table in <code>coded_data\$deltas</code> .
time_limit	Maximum episode-window length in seconds when a frame constraint is applied.
time_limit_frames	Optional maximum episode-window length in frames. If supplied, this takes precedence over <code>time_limit</code> .
constraint_method	Character scalar controlling how the reaction window ends. "strict" uses the earlier of the observed episode end and the requested time limit. "episode" uses the observed episode end and ignores <code>time_limit</code> . "loose" uses the later of the observed episode end and the requested time limit. "frames" uses only the requested time limit and ignores the observed episode end. Default is "episode".
exclude_start	Minimum delay, in seconds from the denominator episode start, before numerator <code>delta == 1</code> values count as a reaction.
exclude_start_frames	Optional minimum delay in frames. If supplied, this takes precedence over <code>exclude_start</code> .
fps	Frames per second used to convert between seconds and frames when <code>coded_data\$metadata\$fps</code> is unavailable.
subject_names	Character vector of subject labels to compare. If NULL, all unique subjects in <code>coded_data\$coding</code> are used.
exclude_emotions	Character vector of emotions to exclude from the denominator calculation. Default is "neutral".

minimum_threshold

Numeric scalar in $[0, 1]$. Denominator episodes are kept only when at least this proportion of numerator frames in the constrained window have non-missing values. Default is 0.

Value

A data.table with columns id, denominator, numerator, emotion, run_id, start_frame, end_frame, n_frames, present_prop, and reaction.

See Also

[reaction_rate\(\)](#)

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_df)
reaction_rate_by_episode(coded_data)

## End(Not run)
```

shared_synchronous_episodes

Map shared synchronous episodes

Description

Maps pairs of same-emotion episodes that overlap while both subjects are in state. Each unordered pair of source episodes is returned once. A source episode may therefore occur in multiple rows when it overlaps more than one episode of the other subject.

Usage

```
shared_synchronous_episodes(
  coded_data,
  episodes = NULL,
  subject = "subject",
  id = "id",
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  fps = 30L,
  missing_threshold = 0,
  exclude_emotions = "neutral"
)
```

Arguments

<code>coded_data</code>	Output from <code>convert_to_episodes()</code> .
<code>episodes</code>	Optional replacement episode data frame. It must contain the selected ID and subject columns plus emotion, run_id, start_frame, end_frame, and max_value. Frame bounds are inclusive.
<code>subject</code>	Character scalar giving the column in <code>coded_data\$coding</code> and <code>coded_data\$episodes</code> that identifies the subject. Default is "subject".
<code>id</code>	Character scalar giving the column in <code>coded_data\$coding</code> and <code>coded_data\$episodes</code> that identifies the case or dyad. Default is "id".
<code>time_limit</code>	Maximum episode-window length in seconds. NULL is allowed when <code>time_limit_frames</code> is supplied or when <code>constraint_method</code> is "episode". Default is 3.
<code>time_limit_frames</code>	Optional maximum episode-window length in frames. If supplied, this takes precedence over <code>time_limit</code> .
<code>constraint_method</code>	Character scalar controlling how the episode window ends: "strict", "episode", "loose", or "frames". Default is "episode".
<code>fps</code>	Frames per second used to convert <code>time_limit</code> to frames. Default is 30.
<code>missing_threshold</code>	Numeric scalar in $[0, 1]$. Denominator and numerator episodes are kept only when the comparison subject is present for at least this proportion of frames within the episode. Default is 0.
<code>exclude_emotions</code>	Character vector of emotions to exclude from the denominator calculation. Default is "neutral".

Value

A data.table with episode-pair identifiers, their inclusive shared frame interval, each source episode's maximum value, the larger maximum, and their combined value.

See Also

[synchrony_by_episode\(\)](#)

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_data)
shared_synchronous_episodes(coded_data)

## End(Not run)
```

synchrony

*Calculate synchrony from converted episodes***Description**

Calculate synchrony from converted episodes

Usage

```
synchrony(
  coded_data,
  subject = "subject",
  id = "id",
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  fps = 30L,
  missing_threshold = 0,
  exclude_emotions = "neutral",
  episodes = NULL
)
```

Arguments

<code>coded_data</code>	Output from <code>convert_to_episodes()</code> .
<code>subject</code>	Character scalar giving the column in <code>coded_data\$coding</code> and <code>coded_data\$episodes</code> that identifies the subject. Default is "subject".
<code>id</code>	Character scalar giving the column in <code>coded_data\$coding</code> and <code>coded_data\$episodes</code> that identifies the case or dyad. Default is "id".
<code>time_limit</code>	Maximum episode-window length in seconds. NULL is allowed when <code>time_limit_frames</code> is supplied or when <code>constraint_method</code> is "episode". Default is 3.
<code>time_limit_frames</code>	Optional maximum episode-window length in frames. If supplied, this takes precedence over <code>time_limit</code> .
<code>constraint_method</code>	Character scalar controlling how the episode window ends: "strict", "episode", "loose", or "frames". Default is "episode".
<code>fps</code>	Frames per second used to convert <code>time_limit</code> to frames. Default is 30.
<code>missing_threshold</code>	Numeric scalar in $[0, 1]$. Denominator and numerator episodes are kept only when the comparison subject is present for at least this proportion of frames within the episode. Default is 0.
<code>exclude_emotions</code>	Character vector of emotions to exclude from the denominator calculation. Default is "neutral".
<code>episodes</code>	Optional replacement episode data frame. It must contain the selected ID and subject columns plus emotion, run_id, start_frame, and end_frame. Frame bounds are inclusive. A <code>synchrony_by_episode()</code> result is also accepted, using its denominator column as the episode subject.

Value

A data.table with columns id, denominator, numerator, emotion, n_episodes, and synchrony.

See Also

[synchrony_by_episode\(\)](#)

Examples

```
library(data.table)

coding <- data.table(
  id = rep(1L, 6),
  subject = rep(c("teen", "parent"), each = 3),
  emotion = "happy",
  video_time = rep(1:3, 2),
  value = c(0.1, 0.2, 0.3, 0.1, 0.2, 0.3),
  in_state = c(FALSE, TRUE, FALSE, FALSE, FALSE, TRUE),
  run_id = c(1L, 1L, 1L, 2L, 2L, 2L)
)
episodes <- data.table(
  id = 1L,
  subject = c("teen", "parent"),
  emotion = "happy",
  run_id = c(1L, 2L),
  start_frame = c(2L, 3L),
  end_frame = c(2L, 3L)
)
coded_data <- structure(
  list(coding = coding, episodes = episodes),
  class = c("fr_coding", "list")
)
synchrony(coded_data, subject = "subject", id = "id", missing_threshold = 0)
```

synchrony_by_episode *Calculate synchrony by denominator episode*

Description

Calculate synchrony by denominator episode

Usage

```
synchrony_by_episode(
  coded_data,
  subject = "subject",
  id = "id",
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  fps = 30L,
  missing_threshold = 0,
  exclude_emotions = "neutral",
```

```
    episodes = NULL
  )
```

Arguments

<code>coded_data</code>	Output from <code>convert_to_episodes()</code> .
<code>subject</code>	Character scalar giving the column in <code>coded_data\$coding</code> and <code>coded_data\$episodes</code> that identifies the subject. Default is "subject".
<code>id</code>	Character scalar giving the column in <code>coded_data\$coding</code> and <code>coded_data\$episodes</code> that identifies the case or dyad. Default is "id".
<code>time_limit</code>	Maximum episode-window length in seconds. NULL is allowed when <code>time_limit_frames</code> is supplied or when <code>constraint_method</code> is "episode". Default is 3.
<code>time_limit_frames</code>	Optional maximum episode-window length in frames. If supplied, this takes precedence over <code>time_limit</code> .
<code>constraint_method</code>	Character scalar controlling how the episode window ends: "strict", "episode", "loose", or "frames". Default is "episode".
<code>fps</code>	Frames per second used to convert <code>time_limit</code> to frames. Default is 30.
<code>missing_threshold</code>	Numeric scalar in $[0, 1]$. Denominator and numerator episodes are kept only when the comparison subject is present for at least this proportion of frames within the episode. Default is 0.
<code>exclude_emotions</code>	Character vector of emotions to exclude from the denominator calculation. Default is "neutral".
<code>episodes</code>	Optional replacement episode data frame. It must contain the selected ID and subject columns plus emotion, run_id, start_frame, and end_frame. Frame bounds are inclusive. A <code>synchrony_by_episode()</code> result is also accepted, using its denominator column as the episode subject.

Details

When `coded_data` contains metadata, its `metadata$fps` value is used by the synchrony calculations, including when `fps` is supplied.

Value

A `data.table` with columns `id`, `denominator`, `numerator`, `emotion`, `run_id`, `start_frame`, `end_frame`, `present_prop`, and `synchrony`.

See Also

[synchrony\(\)](#)

Examples

```
library(data.table)

coding <- data.table(
  id = rep(1L, 6),
  subject = rep(c("teen", "parent"), each = 3),
```

```

    emotion = "happy",
    video_time = rep(1:3, 2),
    value = c(0.1, 0.2, 0.3, 0.1, 0.2, 0.3),
    in_state = c(FALSE, TRUE, FALSE, FALSE, FALSE, TRUE),
    run_id = c(1L, 1L, 1L, 2L, 2L, 2L)
  )
  episodes <- data.table(
    id = 1L,
    subject = c("teen", "parent"),
    emotion = "happy",
    run_id = c(1L, 2L),
    start_frame = c(2L, 3L),
    end_frame = c(2L, 3L)
  )
  coded_data <- structure(
    list(coding = coding, episodes = episodes),
    class = c("fr_coding", "list")
  )
  synchrony_by_episode(coded_data, subject = "subject", id = "id", missing_threshold = 0)

```

synchrony_moments_pipeline

Create synchrony moments and video clips from FaceReader outputs

Description

Recursively discovers videos and FaceReader detailed TXT/XLSX exports below a folder. Each export is matched to a video using the basename in its FaceReader Filename metadata, ignoring case, directories, and file extensions. Videos without matching detailed exports are skipped with a message. Each matched video must have exactly two detailed outputs with distinct, stable Participant Name values.

Usage

```

synchrony_moments_pipeline(
  inpath,
  output_dir = inpath,
  video_pattern = "\\.(mp4|mov|mkv|avi)$",
  video_path = NULL,
  subject_from_filename = FALSE,
  verbose = FALSE,
  T_up = 0.2,
  T_down = 0.1,
  delta = 0.1,
  delta_window = 0.2,
  min_dur_sec = 0.1,
  consecutive_missing = 150L,
  fps = 30L,
  cores = 0L,
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",

```

```

missing_threshold = 0,
exclude_emotions = "neutral",
n = 10L,
emotion = "happy",
optimised_subject = "both",
only_synchronies = TRUE,
buffer = 0,
buffer_units = c("seconds", "frames"),
output = c("zip", "folder"),
overwrite = FALSE,
ffmpeg = "ffmpeg"
)

```

Arguments

<code>inpath</code>	Existing directory containing videos and FaceReader exports.
<code>output_dir</code>	Optional root directory for clip outputs. Defaults to <code>inpath</code> .
<code>video_pattern</code>	Case-insensitive regular expression for video filenames.
<code>video_path</code>	Optional existing video path. When supplied, all detailed exports are paired to this one video instead of their header filenames.
<code>subject_from_filename</code>	Use the FaceReader export filename as the subject when its Participant Name column is absent.
<code>verbose</code>	Whether to list videos with and without matching FaceReader detailed exports. Videos without matching detailed exports are always reported as skipped.
<code>T_up, T_down, delta, delta_window, min_dur_sec, consecutive_missing, fps, cores</code>	Arguments passed to <code>convert_to_episodes()</code> .
<code>time_limit, time_limit_frames, constraint_method, missing_threshold, exclude_emotions</code>	Arguments passed to <code>shared_synchronous_episodes()</code> .
<code>n, emotion, optimised_subject, only_synchronies, buffer, buffer_units, output, overwrite, ffmpeg</code>	Arguments passed to <code>export_shared_synchrony_clips()</code> .

Value

A `synchrony_moments_pipeline` list containing discovery manifest, validated videos, and named per-video results.

Examples

```

## Not run:
synchrony_moments_pipeline("path/to/study-folder", output_dir = "clips")

## End(Not run)

```

to_seconds

Convert FaceReader time strings to seconds

Description

Parses time values formatted as hh:mm:ss or hh:mm:ss.s and returns the equivalent number of seconds. Fractional seconds are truncated or padded to the number of digits specified by digits.

Usage

```
to_seconds(x, digits = 3L)
```

Arguments

x	Character vector of time strings.
digits	Number of fractional digits to retain after the decimal point. Set to 0 to discard fractional seconds. Default is 3.

Value

A numeric vector of seconds.

Examples

```
## Not run:
to_seconds(c("00:00:10.000", "00:00:10.500"))

## End(Not run)
```

Index

`add_delta_column`, [2](#)

`convert_to_episodes`, [6](#)
`convert_to_episodes()`, [16](#), [26](#)
`convertFRDirectory`, [3](#)
`convertFRExcelFiles`, [4](#)
`convertFRFiles`, [5](#)

`delta`, [8](#)
`delta_episodes`, [9](#)

`export_shared_synchrony_clips`, [9](#)
`export_shared_synchrony_clips()`, [26](#)
`extract_id_subject_metadata`, [11](#)
`extract_subject_id_metadata`, [12](#)

`janitor::clean_names()`, [13](#)

`loadFRfile`, [13](#)
`locf`, [14](#)

`map_paths`, [15](#)

`negative_controls`, [15](#)

`parse_time_to_frame`, [17](#)

`reaction_rate`, [17](#)
`reaction_rate()`, [20](#)
`reaction_rate_by_episode`, [19](#)
`reaction_rate_by_episode()`, [18](#)
`readr::read_csv()`, [13](#)

`shared_synchronous_episodes`, [20](#)
`shared_synchronous_episodes()`, [10](#), [26](#)
`synchrony`, [22](#)
`synchrony()`, [24](#)
`synchrony_by_episode`, [23](#)
`synchrony_by_episode()`, [15](#), [16](#), [21](#), [23](#)
`synchrony_moments_pipeline`, [25](#)

`to_seconds`, [27](#)